Reject Inference

When a lender builds a credit scorecard, it only observes repayment outcomes for **accepted applicants**. Rejected applicants never receive a loan, so their future default/non-default outcome is unknown.

This creates **sample selection bias** because the model is trained on a population that is systematically different from the population to which it will be applied. Reject inference attempts to estimate ("infer") the likely outcomes of rejected applicants and include them in model development.

Ref: https://pmc.ncbi.nlm.nih.gov/articles/PMC9041715/

1. Hard Cut-off (Simple Augmentation)

How it works

1. Build a model using accepted applicants only.
2. Score rejected applicants.
3. Choose a probability threshold.
4. Assign:
    - Above threshold → Bad
    - Below threshold → Good
5. Rebuild the model using accepted + inferred rejects.

Example:

Suppose:

Applicant	**Predicted PD**
Reject A	8%
Reject B	15%
Reject C	42%
Reject D	65%

Using a 40% cutoff:

Applicant	Assigned Outcome
A	Good
B	Good
C	Bad
D	Bad

The inferred labels are treated as actual observations in the next model.

Pros

- Very easy to implement.
- Easy to explain to management and regulators.
- Computationally inexpensive.
- Works reasonably well when score separation is strong.

Cons

- Creates artificial certainty.
- A borrower with 39.9% PD becomes "Good" while 40.1% becomes "Bad".
- Errors in the initial model propagate directly.
- Often produces unstable results.

Best use case

- Small portfolios.
- Quick benchmark analysis.
- Initial reject inference testing.

Ref: https://www.ariclabarr.com/credit-modeling/part_4_reject.html?

2. Fuzzy Augmentation

How it works

Instead of assigning a single label, each reject contributes to both classes using probabilities.

Example:

Applicant has:

PD = 30%

Create two records:

Record	Weight
Good	70%
Bad	30%

The model is retrained using weighted observations.

Example

Reject applicant:

- Predicted PD = 25%

Create:

Status	Weight
Good	0.75
Bad	0.25

Another applicant:

- PD = 80%

Create:

Status	Weight
Good	0.20
Bad	0.80

Pros

- Uses probability information efficiently.
- Avoids arbitrary hard cutoffs.
- Reflects uncertainty more realistically.
- Typically smoother than hard cutoff.

Cons

- More difficult to explain.
- Doubles the dataset size.
- Strongly depends on calibration of the original model.
- Limited theoretical justification; some researchers argue it should not outperform the financed-only model under certain assumptions.

Best use case

- Large datasets.
- Organizations comfortable with weighted modeling.

3. Parceling

How it works

Parceling combines score banding and label assignment.

1. Score rejected applicants.
2. Create score bands (parcels).
3. Estimate bad rates within each band.
4. Assign good/bad labels proportionally within each band.

Example

Accepted population:

Score Band	Observed Bad Rate
700-750	5%
650-700	10%
600-650	20%

Assume rejects are riskier and multiply bad rates by 2:

Score Band	Inferred Reject Bad Rate
700-750	10%
650-700	20%

Score Band	Inferred Reject Bad Rate
600-650	40%

If 1,000 rejects fall into the 600-650 band:

- 400 assigned Bad
- 600 assigned Good

Randomly or proportionally.

Pros

- Widely used in banking.
- More realistic than hard cutoff.
- Preserves score distribution.
- Flexible to business judgment.

Cons

- Requires subjective assumptions.
- Results depend heavily on chosen inflation factors.
- Different analysts can obtain very different models.
- Difficult to validate empirically.

Best use case

- Traditional scorecard environments.
- Basel/IFRS risk modeling teams with expert knowledge.

Ref: https://blogs.sw.siemens.com/rapidminer/credit-scoring-series-part-six-segmentation-and-reject-inference/

4. Proportional Assignment

How it works

Rejects are randomly assigned good/bad status according to an assumed bad rate.

Example

Accepted portfolio bad rate:

10%

Assume rejects are 3× riskier.

Reject bad rate:

30%

For 10,000 rejects:

- 3,000 randomly labeled Bad
- 7,000 randomly labeled Good

No individual applicant scoring required.

Pros

- Very simple.
- Quick to implement.
- No need for complex scoring.

Cons

- Ignores individual applicant characteristics.
- Low predictive value.
- Can inject large amounts of noise.

Best use case

- Baseline comparison.
- Sensitivity analysis.

5. Reweighting / Augmentation

How it works

Instead of assigning labels, accepted applicants receive weights so they represent the entire applicant population.

Suppose:

Segment	Acceptance Rate
High Income	80%
Low Income	20%

Low-income borrowers are underrepresented.

Weight:

Weight=1Acceptance RateWeight=\frac{1}{Acceptance\ Rate}Weight=Acceptance Rate1

Therefore:

Segment	Weight
High Income	1.25
Low Income	5.00

The model learns from accepts while compensating for selection bias.

Pros

- No artificial labels.
- Statistically cleaner.
- Easier governance.
- Often preferred by model validators.

Cons

- Requires modeling the acceptance mechanism.
- Sensitive to incorrect weight estimation.
- Can create extreme weights.

Best use case

- Regulatory environments.
- Institutions with robust acceptance-process data.

6. Self-Learning (Semi-Supervised Reject Inference)

How it works

1. Train model on accepts.
2. Score rejects.
3. Assign labels to most confident rejects.
4. Retrain model.
5. Repeat.

Example

Iteration 1:

	Reject PD
A	2%
B	5%
C	50%
D	95%

Only:

- A labeled Good
- D labeled Bad

Add to training data.

Retrain.

Repeat until convergence.

Pros

- Uses unlabeled data effectively.
- Often better than simple augmentation.
- Can improve discrimination.

Cons

- Label errors accumulate.
- Requires careful monitoring.
- More computationally intensive.

Best use case

- Large-scale ML credit scoring.
- Digital lenders and fintechs.

7. Deep Generative / Bayesian Reject Inference

How it works

Uses probabilistic generative models to jointly model:

- Accepted applicants
- Rejected applicants
- Repayment outcomes

The model estimates hidden outcomes for rejects as latent variables.

Example

Instead of assigning:

Reject A = Good

the model estimates:

P(Good)=0.82P(Good)=0.82P(Good)=0.82

and incorporates uncertainty directly into training.

Pros

- Captures complex relationships.
- Handles non-linear interactions.
- Strong research performance.
- Better treatment of uncertainty.

Cons

- Hard to explain.
- Significant computation.
- Difficult validation.
- Regulatory concerns regarding interpretability.

Best use case

- Advanced fintechs.
- Research environments.
- Challenger model development.

Comparison Summary

Technique	Complexity	Explainability	Typical Performance	Main Weakness
Hard Cut-off	Low	High	Moderate	Artificial labels
Fuzzy Augmentation	Medium	Medium	Moderate-High	Depends on initial model
Parceling	Medium	High	Moderate-High	Subjective assumptions
Proportional Assignment	Low	High	Low	Ignores applicant characteristics
Reweighting	Medium	High	Moderate	Requires acceptance model
Self-Learning	High	Medium	High	Error propagation
Deep Generative Models	Very High	Low	High	Governance and explainability

Practical Industry View

Many banks still use **Parceling** and **Fuzzy Augmentation** because they fit traditional scorecard development processes and are relatively easy to explain. However, academic research has repeatedly shown that **no reject inference method consistently outperforms all others across datasets**, and gains are often modest. Some institutions therefore prefer **reweighting** or collecting additional outcome data (e.g., through bureau performance monitoring) rather than relying heavily on reject inference.

Key References

1. Ehrhardt, A. et al. *Reject Inference Methods in Credit Scoring*, Journal of the Royal Statistical Society (2021).
2. MathWorks Documentation: *Use Reject Inference Techniques with Credit Scorecards*.
3. Altair Credit Scoring Series – *Segmentation and Reject Inference*.
4. Labarr, A. *Credit Modeling – Reject Inference Techniques*.
5. Kozodoi et al. *Shallow Self-Learning for Reject Inference in Credit Scoring* (2019).

6. Mancisidor et al. *Deep Generative Models for Reject Inference in Credit Scoring* (2019).